

CodeClash — Presentation Speaking Script

FSJP Mini Project • Computer Engineering (2026-27) • K. C. College of Engineering, Thane

Slide 1: Title Slide

~20 sec

Context: Title, College details, Group: Prince Dubey, Krish Jadhav, Abhishek Yadav, Shivam Yadav, Guide: Prof. Shweta Gaikwad

Script: "Good morning respected teachers and fellow students. Today, our team—Prince, Abhishek, Shivam, and myself, Krish—under the guidance of Prof. Shweta Gaikwad, is excited to present our FSJP project: **CodeClash — A Real-Time 1v1 Competitive Coding Arena.**"

Slide 2: Outline of Presentation

~15 sec

Context: Presentation roadmap from Introduction to System Architecture & Methodology

Script: "Here is the brief roadmap of today's presentation. We will walk you through the core problem we identified, the architecture of our platform, our full-stack implementation using Spring Boot and Java, and how CodeClash bridges the gap in modern coding practice."

Slide 3: Introduction

~30 sec

Context: CodeClash platform overview + 6 core highlights (Spring Boot, Cloud, Judging, Elo, Auth, MySQL)

Script: "Most of us practice Data Structures & Algorithms on platforms like LeetCode, but the experience is largely solitary. **CodeClash** transforms DSA practice into a real-time head-to-head battle. Two players join a private room, receive the exact same problem with a countdown timer, write Java solutions, and are evaluated instantly by a sandboxed judge engine. The winner is determined by correctness and speed, and their official Elo rating updates dynamically after each match."

Slide 4: Problem Statement

~30 sec

Context: No real-time 1v1 practice, remote partners cannot practice together, lack of peer skill tracking

Script: "When analyzing current platforms, we observed three major challenges:

1. Platforms like LeetCode and Codeforces focus either on solo grinding or scheduled global contests—there is no lightweight way to do an instant 1v1 battle with a friend.
2. Study partners are not always on the same local network, which is why a 24/7 cloud-accessible platform is needed.
3. Without direct peer-to-peer competition and rating ladders, students often lose motivation during daily practice."

Slide 5: Literature Survey

~30 sec

Context: Comparison table: LeetCode, Codeforces, HackerRank vs CodeClash

Script: "In our literature survey, we compared leading platforms: *LeetCode* has a vast problem set, but lacks spontaneous live 1v1 dueling. *Codeforces* hosts rated rounds, but they require strict scheduling. *HackerRank* is tailored for recruitment tests rather than casual peer rivalry. **CodeClash fills this gap** by providing instant room-based matchmaking, real-time opponent telemetry, automated test grading, and a Chess-style Elo ranking system."

Slide 6: Aim and Objectives

~25 sec

Context: Core aim + Primary & Secondary objectives

Script: "Our primary aim was to engineer a robust, cloud-hosted 1v1 battle platform. Key objectives achieved include: 6-character private room matchmaking; a sub-second sandboxed compiler and judge engine with watchdog protection against infinite loops; standard K=32 Elo ranking calculations and a global leaderboard; and secure BCrypt authentication with zero local installation required."

Slide 7: System Architecture

~35 sec

Context: Data Flow: Browser -> Spring Boot REST -> Code Execution Engine -> MySQL (Render Cloud)

Script: "Here is our System Architecture: Both players connect from their browsers to our **Spring Boot REST backend** deployed on Render. When a duel begins, the backend distributes the same problem and synchronized timer. As code is submitted, our **Code Execution Engine** executes the solution in an isolated sandbox against hidden test cases. Results, test verdicts, and newly calculated Elo ratings are immediately persisted to the **MySQL database** and synced back to the frontend."

Slide 8: Methodology & Tech Stack

~30 sec

Context: Full-Stack Java: Spring Boot 3.3.4, Java 21/25, JPA/Hibernate, BCrypt, HTML5/CSS, Docker

Script: "For our technology stack: **Backend:** Built on Spring Boot 3.3 and Java 21/25 with Spring Data JPA and Hibernate for robust ORM mapping. **Security:** BCrypt hashing for credential protection. **Frontend:** Modern responsive HTML5, custom CSS design system with Monaco Editor integration. **Execution & Deployment:** Multi-stage Docker containerization deployed on Render with an embedded/MySQL database pipeline."

Slide 9: References

~10 sec

Context: Spring Boot docs, Judge0 CE, MySQL reference, Elo Rating methodology, GitHub repository

Script: "These are the technical references and platform documentations that guided our design, including Spring Boot, Judge0, and the standard Elo rating framework."

Slide 10: Conclusion & Q/A

~15 sec

Context: Thank You slide + Opening floor for teacher questions

Script: "In conclusion, CodeClash brings gamification and friendly rivalry into daily programming practice. Thank you for your time, and we are now open to any questions."

■ Teacher Q&A; Cheat Sheet (Prepared Answers)

Q1: How does your code execution / judging engine ensure security and prevent infinite loops?

Answer: We compile and run Java code in isolated subprocesses with restricted heap memory (-Xmx128m) and enforce a strict 3000ms watchdog timer per test case to abort infinite loops or blocking calls before grading.

Q2: How is the Elo Rating calculated for players?

Answer: We implement the standard Chess K=32 Elo algorithm. If a lower-rated player defeats a higher-rated favorite, they earn a larger rating bonus (+24 to +30), ensuring a fair and competitive leaderboard.

Q3: Is the application live and accessible from anywhere?

Answer: Yes, it is fully containerized with Docker and hosted 24/7 on Render at <https://codeclash-p5dd.onrender.com>, so anyone with a browser on desktop or mobile can duel instantly.